

Internship Report About Data Analysis Concepts

Name: Galip Alperen Aydınlık

Topic: Data Preparation and Classification

1.Introduction

1.1 What is Data Preparation

Data Preparation means cleaning and transforming raw data into structured format so the data can be easily trained and analyzed.

1.2 What is Classification

Classification is ML technique that allows us to categorize the data that we obtain.

Examples can be given such as Student Pass / Fail detector. Depending on the students information (study hours, midterm, final, quiz, result), we can analyze these values and find that student passed or failed.

1.3 Accuracy

Accuracy is proportion of all classifications that were correct, whether positive or negative.

2.Tools Used

- Python (PyCharm)
- Pandas library
- Scikit – learn
- MySQL

3.Example Projects

I have implemented few projects next page that explains all these topics briefly.

3.1 Iris Classification Project

Iris Dataset is one of the most known and popular Machine Learning dataset from 1936 often used to test Machine Learning Algorithms.

It uses simple classification model to predict the flowers species using it's information such as sepal length, sepal width and petal width.

Flowers species are: Setosa, Versicolor, Virginica.

We can import the Iris Dataset from scikit-learn library.

By using scikit-learn, we can also import train_test_split which can be used to train model, KNeighborsClassifier and accuracy score which I will explain later on.

Here is the code below:

```
main.py x
1  from sklearn import datasets
2  from sklearn.model_selection import train_test_split # this is used to train the model
3  from sklearn.neighbors import KNeighborsClassifier
4  from sklearn.metrics import accuracy_score
5  iris = datasets.load_iris()
6
7  X = iris.data # Sepal length, sepal width, petal length, petal width
8  Y = iris.target # 0 for Setosa, 1 for Versicolor, 2 for Virginica
9
10 print("Feature Names:\n", iris.feature_names)
11 print("Target Names:\n", iris.target_names)
12
13 print("X Shape:\n", X.shape)
14 print("Y Shape:\n", Y.shape)
15
16 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.3, random_state = 0)
17 # Used the 30% of data to testing
18
19 print("X_train Shape:\n", X_train.shape)
20 print("X_test Shape:\n", X_test.shape)
21 print("Y_train Shape:\n", Y_train.shape)
22 print("Y_test Shape:\n", Y_test.shape)
23
24 model = KNeighborsClassifier(3)
25 model.fit(X_train, Y_train) # training the models
26 y_pred = model.predict(X_test)
27
28 accuracy = accuracy_score(Y_test, y_pred)
29
30 print("Predictions:\n", y_pred)
31 print("Actual Values:\n", Y_test)
32 print("Accuracy:\n", accuracy)
```

At start, we used sklearn library and imported KneightborClassifier. KNN predicts the class of a new data point by comparing it with the closest data points in the training dataset. The model checks the nearest neighbors and assigns the class that appears most frequently among them.

from sklearn.metrics import accuracy_score : this part lets us to use accuracy rate which showed later on code.

After we taked the Iris Dataset and print it's information, we used train_test_split function to test data.

As model, we used KNN (looks at its 3 neighbors), then trained the models and created predictions.

Accuracy variable assigned to accuracy_score() function which is the accuracystate of the model.

At last, predictions, actual values, and the accuracy of these values are printed.

```
print("Predictions:\n", y_pred)
print("Actual Values:\n", Y_test)
print("Accuracy:\n", accuracy)
```

```
Accuracy:
0.9777777777777777
```

3.2 Student Pass / Fail Predictor

This projects aims to predict students passed or failed using students given information which is study_hours, attendance, midterm and final.

Its a classification project so the output have 2 options:

Result = 1 (Pass)

Result = 0 (Fail)

This project DecisionTreeClassifier from sklearn and make prediction using student data.

For this project, pandas library is also implemented.

Code is in the next page:

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier # it uses if and else questions to understand the result
4 from sklearn.metrics import accuracy_score
5
6 data = {
7     "name": ["Ali", "Ayşe", "Mehmet", "Zeynep", "Can", "Elif", "Burak", "Deniz", "Ece", "Mert"],
8     "study_hours": [5, 2, 8, 1, 4, 6, 3, 7, 2, 9],
9     "attendance": [80, 50, 90, 40, 70, 85, 60, 95, 45, 98],
10    "midterm": [60, 40, 75, 30, 55, 65, 45, 80, 35, 90],
11    "final": [70, 45, 85, 35, 60, 75, 50, 88, 40, 95],
12    "result": [1, 0, 1, 0, 1, 1, 0, 1, 0, 1]
13 }
14
15 df = pd.DataFrame(data)
16 print(df.head()) # prints first 5 students grades
17 print("\n")
18
19 print(df.iloc[-1]) # prints the last students grades
20 print("\n")
21
22 x = df[["study_hours", "attendance", "midterm", "final"]]
23 y = df["result"]
24
25 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.4, random_state=0)
26
27 model = DecisionTreeClassifier()
28 model.fit(x_train, y_train)
29
30 y_pred = model.predict(x_test)
31
32 accuracy = accuracy_score(y_test, y_pred)
33
34 print("Predictions: ", y_pred)
35 print("Real Values: ", list(y_test))
36 print("Accuracy: ", accuracy)

```

Different than Iris Flower Project, pandas library is added to create DataFrame for the given data variable.

DecisionTreeClassifier used because the goal is to predict whether a student passes or fails. This is a classification problem because the output has two possible classes: 1 for pass and 0 for fail.

Just like Iris Flower Project accuracy_score and train_test_split is added.

At first, data is given which includes names of the students, study_hours, attendance, midterm and final results.

By using pandas library, we created variable named df and stored information in it and then df.head() function used to print the first part of the data given.

I also added an df.iloc[] from pandas library that prints the location you choose.

X and y variables used to separate data information.

Again, train_test_split used to model should be tested on data it has not seen before.

%40 of data used for testing. Since we have 10 students, 6 students are used for training 4 students are used for testing.

DecisionTreeClassifier used as training model which fits the goal of the code.

We trained the model, made predictions, find accuracy rate of these predictions and we printed them.

```
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)

print("Predictions: ", y_pred)
print("Real Values: ",list(y_test))
print("Accuracy: ", accuracy)
```

Overall this project is very similar to Iris Flower Project but pandas library used additionally.

3.3 Finding Total Treatments Cost and Count Using MySQL Dataset

Aim of the project is to find each doctors total treatment cost and count how many treatments they did.

This project doesnt have any model training, prediction and accuracy rate unlike others.

Its implemented to show how MySQL database can connect to PyCharm.

To import data from MySQL, we can use mysql-connector-python library.

Then we use import mysql.connector and create variable called mydb which we define our MySQL information such as our name, password and or database's name.

Code is at next page:

```

main.py x
1 import mysql.connector
2
3 mydb = mysql.connector.connect(
4     host="localhost",
5     user="root",
6     passwd= "alperen",
7     port= 3306,
8     database= "hospital"
9 )
10
11 my_cursor = mydb.cursor()
12
13 query = """
14 SELECT
15     d.doctor_id,
16     COUNT(t.treatment_id) AS treatment_count,
17     SUM(t.cost) AS total_charge
18 FROM doctors d
19 JOIN treatments t
20     ON d.doctor_id = t.doctor_id
21 GROUP BY d.doctor_id
22 ORDER BY total_charge DESC;
23 """
24
25
26 my_cursor.execute(query)
27
28 treatments = my_cursor.fetchall()
29
30 for treatment in treatments:
31     doctor_id = treatment[0]
32     treatment_count = treatment[1]
33     total_charge = treatment[2]
34
35     print(f"Doctor ID: {doctor_id}")
36     print(f"Treatment Count: {treatment_count}")
37     print(f"Total Charge: {total_charge}")
38     print("-----")
39
40 my_cursor.close()
41 mydb.close()

```

The execute() function is used to run the SQL query through the cursor. After the query is executed, the fetchall() function retrieves all rows returned by the query and stores them in the treatments variable. These results are then printed one by one using a loop.

Here is the some part of the output:

```

Doctor ID: 5
Treatment Count: 5
Total Charge: 5430.00
-----
Doctor ID: 10
Treatment Count: 5
Total Charge: 4350.00
-----
Doctor ID: 8
Treatment Count: 5
Total Charge: 4120.00

```

3.4 Filament Print Success Classification Project

Aim of the project is to predict 3D printing process will be successful or not by analyzing filament types, and details about temperature of the filament used.

Just like the Doctor Treatment Project, we used MySQL Database that I created myself in MySQL Workbench.

```
1 • CREATE DATABASE PrinterWorld;
2 • USE PrinterWorld;
3
4 • DROP TABLE FILAMENT;
5
6 • CREATE table FILAMENT (
7     print_id INT AUTO_INCREMENT PRIMARY KEY,
8     filament_type VARCHAR(10),
9     nozzle_temp INT,
10    bed_temp INT,
11    print_speed INT,
12    cooling_fan INT,
13    layer_height FLOAT,
14    success INT
15 );
16
17 • INSERT INTO FILAMENT (filament_type, nozzle_temp, bed_temp, print_speed, cooling_fan, layer_height, success)
18 VALUES
19 ('PLA', 200, 60, 70, 100, 0.20, 1),
20 ('PLA', 205, 55, 90, 100, 0.20, 1),
21 ('PLA', 215, 60, 100, 90, 0.24, 1),
22 ('PLA', 185, 40, 120, 100, 0.20, 0),
23 ('PLA', 230, 70, 110, 100, 0.16, 0),
24 ('PLA', 195, 50, 130, 80, 0.28, 0),
25 ('PETG', 235, 75, 45, 40, 0.20, 1),
26 ('PETG', 240, 80, 55, 35, 0.24, 1),
27 ('PETG', 245, 85, 50, 30, 0.20, 1),
28 ('PETG', 220, 60, 90, 100, 0.20, 0),
29 ('PETG', 250, 70, 100, 100, 0.16, 0).
```

But additionally, pandas library and other sklearn library functions added since this is a classification project.

First part of the code is just printing SQL Database using functions from the previous Doctor Treatment Project.

Code is at the next page:

```
main.py x
1 import mysql.connector # to connect MySQL datasets to PyCharm
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.preprocessing import LabelEncoder
6 from sklearn.metrics import accuracy_score
7
8 mydb = mysql.connector.connect(
9     host="localhost",
10    user="root",
11    passwd= "alperen",
12    port= 3306,
13    database= "printerworld"
14 )
15
16 my_cursor = mydb.cursor()
17
18 query = "SELECT * FROM filament"
19 my_cursor.execute(query)
20
21 rows = my_cursor.fetchall()
22
23 for row in rows: # printing filament table from MySQL
24     print(row)
25
26 data = pd.read_sql(query, mydb)
27 print(data.head())
28 print("\n")
29 print(data.info())
30 print("\n")
31
32 encoder = LabelEncoder()
33 data["filament_type"] = encoder.fit_transform(data["filament_type"]) #convert filament type text to number
34
35 print(data.head())
36 print("\n")
37
38 X = data[["filament_type", "nozzle_temp", "bed_temp", "cooling_fan", "layer_height"]]
39 Y = data["success"]
```

```
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, Y, test_size = 0.4, random_state = 42)
model = DecisionTreeClassifier(random_state = 42) # used decision tree because it's a classification project, also uses same training row everytime

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Prediction: ", y_pred)
print("Accuracy: ", accuracy)
print("Real Values", list(y_test))
```

As you can see, we have added LabelEncoder function that haven't been used in previous projects.

LabelEncoder converts text to numbers because ML models use numbers.

We divided the data to 2 categories named x and y. Then we used the same train_test_split functions just like previous projects to test the data.

As a model, DecisionTreeClassifier used because this is a classification project and we must predict that 3D printing process successfull or not.

End of the code is same with others so doesnt need further explanation.

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Prediction: ", y_pred)
print("Accuracy: ", accuracy)
print("Real Values",list(y_test))
```

3.5 Video Game Success Classification Project

This is the most complex and various project that includes everything we have learned so far.

Its also classification project like others because we have the outputs:

1: Successful Game

2: Unsuccessfull Game

In this project, we have created an csv file and read it using pandas library function read_csv().

Additionaly there are many imported sklearn functions.

There also also missing values at the csv file to show us how Data Preparation works.

Code is at the next page:

```

main.py x games_dataset.txt
1 import pandas as pd
2 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
3 from sklearn.model_selection import train_test_split
4 from sklearn.compose import ColumnTransformer # preparation method
5 from sklearn.pipeline import Pipeline # connects multiple steps together
6 from sklearn.impute import SimpleImputer # fixes missing data
7 from sklearn.ensemble import RandomForestClassifier # for classification
8 from sklearn.preprocessing import OneHotEncoder # converts text categories into numbers so machine understand
9
10 df = pd.read_csv('games_dataset.txt')
11
12 print(df)
13 print("\n")
14
15 print("Missing Values:")
16 print(df.isnull().sum())
17 print("\n")
18
19 x = df.drop(labels="success", axis = 1) # axis = 1 for columns, 0 for rows
20 y = df["success"]
21
22 numeric_features = ["price", "user_rating", "critic_score", "avg_playtime", "review_count"]
23 categorical_features = ["genre", "platform"]
24
25 numeric_transformer = Pipeline(steps=[("imputer", SimpleImputer(strategy="mean"))])
26 # for numeric data, if data is missing, it gives mean of the other datas
27
28 categorical_transformer = Pipeline(steps=[("imputer", SimpleImputer(strategy="most_frequent")),
29                                           ("encoder", OneHotEncoder(handle_unknown="ignore"))
30 ])
31 # if there are missing data, give the most frequently use categorial feature
32 # if the model sees a new category, don't crash
33
34 preprocessor = ColumnTransformer(transformers=[("num", numeric_transformer, numeric_features),
35                                              ("cat", categorical_transformer, categorical_features)
36 ])
37 # combines numerical cleaning and categorial cleaning
38
39 model = Pipeline(steps=[("preprocessor", preprocessor),
40                         ("classifier", RandomForestClassifier(random_state=0))
41 ])
42

```

```

model.fit(x_train, y_train)
y_pred = model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nPredictions: ", y_pred)
print("\nAccuracy: ", accuracy)
print("\nReal Values: ", list(y_test))
print("\nConfusion Matrix: ", confusion_matrix(y_test, y_pred))
print("\nClassification Report: ", classification_report(y_test, y_pred))

```

From sklearn: Pipeline, SimpleImputer, RandomForestClassifier, OneHotEncoder and sklearn.metrics functions (accuracy_score, classification_report, confusion_matrix) have been added to the project.

`print(df.isnull().sum())` is used to show all of the missing values at the csv file.

Data is separated to 2 parts again but this time it implemented little bit different than other projects.

```
x = df.drop("success", axis=1)
```

```
y = df["success"]
```

Variable x has every column except success and y has only the success column.

We also divided the data to numeric features such as price, user rating, etc.. and categorial features such as genre and platform.

After that we used the Pipeline function:

```
numeric_transformer = Pipeline(steps=[("imputer",  
SimpleImputer(strategy="mean"))])
```

This function fixes missing numbers by replacing them with the column average.

For example if we have 4 prices (\$20, \$30, missing, \$40), mean will be 30 $(20+30+40) / 3$

So, the missing value will be \$30.

There is another Pipeline function:

```
categorical_transformer = Pipeline(steps=[("imputer",  
SimpleImputer(strategy="most_frequent")),  
("encoder", OneHotEncoder(handle_unknown="ignore"))  
])
```

Here there are two parts: one is SimpleImputer and the other one is OneHotEncoder.

We used SimpleImputer part to fill the missing text values to most frequently used value

For example lets just say we have a list like this

1.Action, 2.RPG, 3.Action, 4.Missing

Most used genre is Action so the missing genre will be Action.

As for second part, we used the OneHotEncoder function that converts text categories into numbers.

Machine Learning Models cant understand direct words so we have to convert them into numbers.

Example in the next page:

genre
Action
RPG
Strategy

Becomes

genre_Action	genre_RPG	genre_Strategy
1	0	0
0	1	0
0	0	1

handle_unknown="ignore" means that if the model sees a new category, dont crash.

We defined an variable named preprocessor and used ColumnTransformer from sklearn library to combine the prepared numerical and categorial values which we turned to numbers.

We used RandomForestClassifier because RandomForestClassifier uses many decision trees instead of only one and each tree makes a prediction, and the forest chooses the most common answer.

After that we will do the same things we have done in previous project, but added confusion matrix and classification report from sklearn additionally

```
x_train, x_test, y_train, y_test = train_test_split(*arrays: x, y, test_size=0.3, random_state=0)

model.fit(x_train, y_train)
y_pred = model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nPredictions: ", y_pred)
print("\nAccuracy: ", accuracy)
print("\nReal Values: ", list(y_test))
print("\nConfusion Matrix: ", confusion_matrix(y_test, y_pred))
print("\nClassification Report: ", classification_report(y_test, y_pred))
```

4. Conclusion

In this report, I have showed many projects that explains features such as model training, testing data, data preparation, data prediction and accuracy rates.

The main purpose of these projects was to understand how data can be prepared, analyzed, and used for prediction.

I have used many resources such as scikit-learn library, pandas library, PyCharm and MySQL.